Founder Intelligence Briefing Agent

Complete Setup Guide: Google ADK + TinyFish Web Agent + Accelerator Application

📋 **What's Inside**	Phase 0: Environment Setup → Phase 1: ADK Multi-Agent Build → Phase 2: TinyFish Integration → Phase 3: Accelerator Application. Every command is copy-paste ready.

⏱ **Time**	Phase 0–1 completable in one weekend. Phase 2 adds TinyFish web automation in ~3 hours. Accelerator application takes 1 day.

PHASE 0

Environment Setup — Before You Write Any Code

0.1 Install Prerequisites

Open your terminal and run these in order. This works on Mac, Linux, and WSL on Windows.

```
# 1. Check Python version — need 3.10 or higher
$ python3 --version

# 2. If you need to install/upgrade Python (Mac with Homebrew)
$ brew install python@3.12

# 3. Install pipx for clean global tool installs
$ pip install pipx
$ pipx ensurepath

# 4. Install Google ADK globally
$ pip install google-adk

# 5. Verify ADK installed correctly
$ adk --version
```

0.2 Get Your API Keys — All 4 You Need

Key	Where to Get It	Cost	Used For
GOOGLE_API_KEY	aistudio.google.com → Get API Key	Free tier available	Gemini 2.0 Flash (your LLM brain)
TINYFISH_API_KEY	Credits already received — paste from your TinyFish confirmation email	✅ CONFIRMED — Credits Active	Web browsing / competitor research
STRIPE_SECRET_KEY	dashboard.stripe.com → Developers → API Keys	Free (your own account)	Revenue pulse — MRR, churn
GMAIL_CREDENTIALS	console.cloud.google.com → OAuth 2.0 Client ID	Free	Inbox intelligence agent

🔑 **Accelerator Tip**

Request $2,000 in free Google Cloud credits immediately via cloud.google.com/startup (pre-funded category). This covers your Gemini API costs for the full 9-week program.

0.3 Create Your Project Structure

```
# Create your project folder
$ mkdir founder-briefing-agent && cd founder-briefing-agent

# Create a Python virtual environment
$ python3 -m venv venv
$ source venv/bin/activate          # Mac/Linux
# OR: venv\Scripts\activate         # Windows

# Install all dependencies
$ pip install google-adk google-generativeai requests python-dotenv
$ pip install google-auth google-auth-oauthlib google-api-python-client

# Create your folder structure
$ mkdir -p founder_agent/sub_agents
$ touch .env founder_agent/__init__.py founder_agent/agent.py
$ touch founder_agent/sub_agents/__init__.py
```

0.4 Set Up Your .env File

```
# .env — paste your real keys here, never commit this file
GOOGLE_API_KEY=your_google_ai_studio_key_here
TINYFISH_API_KEY=paste_your_confirmed_key_here   # ✅ Credits received
STRIPE_SECRET_KEY=sk_test_your_stripe_key_here

# List your top 3 competitors (comma-separated)
COMPETITOR_LIST=Notion,Linear,Asana

# Your Gmail — used for inbox intelligence
FOUNDER_EMAIL=you@yourcompany.com
```

🔒 **Security**

Add .env to your .gitignore immediately: echo '.env' >> .gitignore — Never push API keys to GitHub.

PHASE 1

Build the Multi-Agent System with Google ADK

1.1 Build the Revenue Agent (Stripe)

Create the file: founder_agent/sub_agents/revenue_agent.py

```
# founder_agent/sub_agents/revenue_agent.py
import os, requests
from google.adk.agents import LlmAgent
from google.adk.tools import FunctionTool
from dotenv import load_dotenv
load_dotenv()

def get_stripe_revenue() -> dict:
    """Fetches MRR and recent customer activity from Stripe."""
    headers = {'Authorization': f'Bearer {os.getenv("STRIPE_SECRET_KEY")}'}
    # Get recent charges (last 7 days)
    charges = requests.get(
        'https://api.stripe.com/v1/charges',
        headers=headers,
        params={'limit': 20, 'created[gte]': int(__import__('time').time())
- 604800}
    ).json()
    total = sum(c['amount'] for c in charges.get('data',[]) if c['paid']) /
100
    return {'weekly_revenue_usd': total, 'recent_charges':
len(charges.get('data',[]))}

revenue_tool = FunctionTool(func=get_stripe_revenue)

revenue_agent = LlmAgent(
    name='revenue_agent',
    model='gemini-2.0-flash',
    description='Analyzes Stripe revenue data and summarizes MRR and
trends.',
    instruction='''
    You are a revenue analyst. Use get_stripe_revenue to fetch data.
    Return ONLY 2 sentences: current weekly revenue and one trend
observation.
    Be specific with numbers. No fluff.
    ''',
```

```
    tools=[revenue_tool],
)
```

1.2 Build the Inbox Intelligence Agent (Gmail)

Create the file: founder_agent/sub_agents/inbox_agent.py

```
# founder_agent/sub_agents/inbox_agent.py
import os
from google.adk.agents import LlmAgent
from google.adk.tools import FunctionTool
from googleapiclient.discovery import build
from google.oauth2.credentials import Credentials

def get_urgent_emails() -> dict:
    """Fetches top 10 unread emails from Gmail and returns subjects +
senders."""
    # Load OAuth token (generated in Step 1.5)
    creds = Credentials.from_authorized_user_file('token.json')
    service = build('gmail', 'v1', credentials=creds)
    results = service.users().messages().list(
        userId='me', q='is:unread', maxResults=10
    ).execute()
    messages = results.get('messages', [])
    email_data = []
    for msg in messages[:5]:
        detail = service.users().messages().get(userId='me',
id=msg['id']).execute()
        headers = {h['name']: h['value'] for h in
detail['payload']['headers']}
        email_data.append({'from': headers.get('From',''), 'subject':
headers.get('Subject','')})
    return {'unread_count': len(messages), 'top_emails': email_data}

inbox_tool = FunctionTool(func=get_urgent_emails)

inbox_agent = LlmAgent(
    name='inbox_agent',
    model='gemini-2.0-flash',
    description='Reads Gmail and surfaces the most urgent emails for
founder review.',
    instruction='''
    You are an executive assistant. Use get_urgent_emails to pull inbox
data.
    List the top 3 emails that need the founder's attention today.
    For each: one line with sender + subject + why it is urgent.
```

```
    Skip newsletters, notifications, and anything obviously low priority.
    ''',
    tools=[inbox_tool],
)
```

1.3 Build the Competitor Radar Agent (TinyFish)

This is where TinyFish Web Agent becomes your superpower. The CompetitorAgent uses TinyFish to actually browse websites — not just search — and extract real intelligence about competitor moves.

Create the file: founder_agent/sub_agents/competitor_agent.py

```
# founder_agent/sub_agents/competitor_agent.py
import os, requests
from google.adk.agents import LlmAgent
from google.adk.tools import FunctionTool
from dotenv import load_dotenv
load_dotenv()

TINYFISH_BASE = 'https://agent.tinyfish.ai/v1/automation/run-sse'
TINYFISH_HEADERS = {
    'X-API-Key': os.getenv('TINYFISH_API_KEY'),
    'Content-Type': 'application/json'
}

def browse_competitor_news(competitor_name: str) -> dict:
    """Uses TinyFish to browse a competitors website and extract recent
updates."""
    response = requests.post(
        TINYFISH_BASE,
        headers=TINYFISH_HEADERS,
        json={
            'url': f'https://{competitor_name.lower()}.com/blog',
            'goal': f'Find any new product launches, pricing changes, or
major announcements
                     from {competitor_name} in the last 30 days. Return as
JSON with keys:
                     headline, date, and significance (high/medium/low).'
        },
        stream=True
    )
    # Parse SSE stream for final COMPLETE event
    result = {}
    for line in response.iter_lines():
```

```
        if line:
            decoded = line.decode('utf-8')
            if '"type": "COMPLETE"' in decoded:
                import json
                data = json.loads(decoded.replace('data: ','').strip())
                result = data.get('resultJson', {})
                break
    return result or {'status': 'no_updates_found', 'competitor':
competitor_name}

def scrape_competitor_pricing(competitor_url: str) -> dict:
    """Uses TinyFish to check if a competitors pricing page has changed."""
    response = requests.post(
        TINYFISH_BASE,
        headers=TINYFISH_HEADERS,
        json={
            'url': competitor_url,
            'goal': 'Extract all pricing tiers, their prices, and key
features. Return as JSON.'
        },
        stream=True
    )
    result = {}
    for line in response.iter_lines():
        if line:
            decoded = line.decode('utf-8')
            if '"type": "COMPLETE"' in decoded:
                import json
                data = json.loads(decoded.replace('data: ','').strip())
                result = data.get('resultJson', {})
                break
    return result

news_tool    = FunctionTool(func=browse_competitor_news)
pricing_tool = FunctionTool(func=scrape_competitor_pricing)

competitor_agent = LlmAgent(
    name='competitor_agent',
    model='gemini-2.0-flash',
    description='Uses TinyFish to browse competitor sites and extract
strategic intelligence.',
    instruction='''
    You are a competitive intelligence analyst.
    Use browse_competitor_news for each competitor in the COMPETITOR_LIST
env var.
    Surface ONLY high or medium significance updates.
    Return max 3 bullet points. Each bullet: competitor name + what changed
+ why it matters.
    If nothing significant found, say: No significant competitor moves in
the last 30 days.
```

```
    ''',
    tools=[news_tool, pricing_tool],
)
```

Why TinyFish Here	Google Search gives you headlines. TinyFish actually navigates the page — it can read behind dynamic content, bypass bot protection, fill forms, and return structured JSON. This means your competitor agent can check a competitor's changelog, pricing page, or job board — not just their blog RSS.

1.4 Build the Coordinator Agent (Main Brain)

Create the file: founder_agent/agent.py — this is your root agent that orchestrates everything.

```
# founder_agent/agent.py
from google.adk.agents import LlmAgent
from google.adk.tools import AgentTool
from .sub_agents.revenue_agent import revenue_agent
from .sub_agents.inbox_agent import inbox_agent
from .sub_agents.competitor_agent import competitor_agent
import datetime

today = datetime.date.today().strftime('%A, %B %d %Y')

# Wrap each sub-agent as a callable tool
revenue_tool    = AgentTool(agent=revenue_agent)
inbox_tool      = AgentTool(agent=inbox_agent)
competitor_tool = AgentTool(agent=competitor_agent)

# The coordinator is the root_agent ADK looks for
root_agent = LlmAgent(
    name='founder_briefing_coordinator',
    model='gemini-2.0-flash',
    description='Chief of staff AI that produces a daily founder
intelligence brief.',
    instruction=f'''
    You are an elite chief of staff briefing a tech founder every morning.
    Today is {today}.

    Your job: call ALL sub-agents, then produce one structured morning
brief.

    BRIEF FORMAT (always follow this exactly):
    ---
    GOOD MORNING BRIEF — {today}
```

```
    REVENUE PULSE
    [2 sentences from revenue_agent]

    INBOX PRIORITIES (Top 3)
    [3 bullets from inbox_agent]

    COMPETITOR RADAR
    [bullets from competitor_agent, or 'All clear.']

    ONE DECISION FOR TODAY
    [State the single most important decision the founder should make
today]
    ---

    Rules:
    - Be direct. No filler. Founders are busy.
    - Call revenue_agent, inbox_agent, and competitor_agent before writing.
    - Never make up data. If an agent fails, note it honestly.
    ''',
    tools=[revenue_tool, inbox_tool, competitor_tool],
)
```

1.5 Set Up Gmail OAuth (One-Time)

This is the only slightly fiddly step. You only do it once and it stores a token.json file.

1. Go to console.cloud.google.com → APIs & Services → Library → enable Gmail API
2. Go to APIs & Services → OAuth consent screen → External → add your email as test user
3. Go to APIs & Services → Credentials → Create Credentials → OAuth 2.0 Client ID → Desktop App → download credentials.json into your project root
4. Run this one-time auth script in your terminal:

```
# Run once to generate token.json
$ python3 -c "
from google_auth_oauthlib.flow import InstalledAppFlow
flow = InstalledAppFlow.from_client_secrets_file(
    'credentials.json',
    ['https://www.googleapis.com/auth/gmail.readonly']
)
creds = flow.run_local_server(port=0)
import json
```

```
with open('token.json', 'w') as f:
    f.write(creds.to_json())
print('token.json saved — you are done!')
"
```

✅ After this step	You will have a token.json file in your project root. The inbox_agent will use this automatically. You never need to log in again unless the token expires.

1.6 Run Your Agent — See It Work

```
# From your project root (make sure venv is active)
$ source venv/bin/activate

# Launch the ADK development UI in your browser
$ adk web

# ADK will print something like:
# → Running at http://localhost:8000
# Open that URL in Chrome

# In the ADK Dev UI:
# 1. Select 'founder_briefing_coordinator' from the dropdown
# 2. Type: 'Give me my morning brief'
# 3. Watch each sub-agent fire in real time in the trace panel
# 4. See your brief appear in ~15-30 seconds
```

🎯 You should see	The coordinator calling revenue_agent → inbox_agent → competitor_agent in sequence, then synthesizing all outputs into a formatted morning brief. The ADK trace panel shows every tool call in real time.

PHASE 2

TinyFish Deep Integration — Power Up Your Web Agent

TinyFish is what makes this agent genuinely powerful. Beyond competitor research, here are 3 more TinyFish integrations to add for accelerator demo impact.

2.1 Add TinyFish Investor Update Scraper

Founders spend 6-8 hours every quarter on board decks. This integration lets your agent pull relevant industry data to auto-draft context for investor updates.

Create: founder_agent/sub_agents/investor_agent.py

```
# founder_agent/sub_agents/investor_agent.py
import os, requests
from google.adk.agents import LlmAgent
from google.adk.tools import FunctionTool

TINYFISH_BASE = 'https://agent.tinyfish.ai/v1/automation/run-sse'
HEADERS = {'X-API-Key': os.getenv('TINYFISH_API_KEY'), 'Content-Type':
'application/json'}

def get_industry_benchmarks(sector: str) -> dict:
    """Uses TinyFish to scrape SaaS/startup benchmark data relevant to our
sector."""
    response = requests.post(
        TINYFISH_BASE, headers=HEADERS,
        json={
            'url': 'https://www.keybanc.com/saas-survey',
            'goal': f'Find median ARR growth rate, NRR, and CAC payback
benchmarks
                     for {sector} SaaS companies. Return as JSON.'
        },
        stream=True
    )
    for line in response.iter_lines():
        if line and '"type": "COMPLETE"' in line.decode():
            import json
            return json.loads(line.decode().replace('data:
','').strip()).get('resultJson',{})
```

```
    return {}

benchmark_tool = FunctionTool(func=get_industry_benchmarks)

investor_agent = LlmAgent(
    name='investor_agent',
    model='gemini-2.0-flash',
    description='Prepares investor update context by scraping industry
benchmarks.',
    instruction='''
    You help founders prep investor updates.
    Use get_industry_benchmarks to pull relevant benchmark data.
    Return 3 benchmark stats the founder can reference in their next
investor update.
    Format: Metric | Industry Benchmark | Why it matters
    ''',
    tools=[benchmark_tool],
)
```

2.2 Add TinyFish Job Board Monitor (Hiring Intelligence)

One of the best leading indicators of a competitor's strategy is who they are hiring. This agent monitors competitor job boards weekly.

```
# Add to competitor_agent.py tools

def monitor_competitor_hiring(competitor_name: str) -> dict:
    """Uses TinyFish to check what roles a competitor is actively hiring
for."""
    response = requests.post(
        TINYFISH_BASE, headers=TINYFISH_HEADERS,
        json={
            'url': f'https://jobs.lever.co/{competitor_name.lower()}',
            'goal': 'List all open job positions grouped by department.
Return as JSON with
                     departments as keys and list of job titles as values.'
        },
        stream=True
    )
    for line in response.iter_lines():
        if line and '"type": "COMPLETE"' in line.decode():
            import json
            return json.loads(line.decode().replace('data:
','').strip()).get('resultJson',{})
    return {'status': 'no_data', 'competitor': competitor_name}
```

```
# Then add to competitor_agent tools list:
hiring_tool = FunctionTool(func=monitor_competitor_hiring)
# competitor_agent = LlmAgent(..., tools=[news_tool, pricing_tool,
hiring_tool])
```

2.3 Deliver the Brief — WhatsApp or Email

The agent is useless if the founder has to open a browser to see it. Add delivery so the brief arrives automatically every morning.

Option A — Email via Gmail API (simplest):

```
# founder_agent/deliver.py
import base64, os
from email.mime.text import MIMEText
from googleapiclient.discovery import build
from google.oauth2.credentials import Credentials

def send_brief_email(brief_text: str):
    creds = Credentials.from_authorized_user_file('token.json')
    service = build('gmail', 'v1', credentials=creds)
    message = MIMEText(brief_text)
    message['to'] = os.getenv('FOUNDER_EMAIL')
    message['subject'] = f'Your Morning Brief'
    encoded = base64.urlsafe_b64encode(message.as_bytes()).decode()
    service.users().messages().send(userId='me', body={'raw':
encoded}).execute()
    print('Brief sent!')
```

Option B — Schedule it daily with cron (Mac/Linux):

```
# Add to crontab: runs at 7am every weekday
$ crontab -e

# Paste this line (update paths to match your system):
0 7 * * 1-5 cd /path/to/founder-briefing-agent && source venv/bin/activate
&& python3 run_brief.py

# run_brief.py — triggers the agent and emails the output
# from google.adk.runners import InMemoryRunner
# from founder_agent.agent import root_agent
```

```
# runner = InMemoryRunner(agent=root_agent)
# result = runner.run('Give me my morning brief')
# send_brief_email(result.text)
```

PHASE 3

TinyFish Accelerator Application — Step by Step

Timeline Reality Check	The accelerator started February 17, 2026. You are applying now (Feb 22). Applications are reviewed every Friday — meaning your next review window is this coming Friday. Move fast.

3.1 TinyFish API Credits — Confirmed ✅

Your TinyFish API credits are confirmed and active. This is a significant unlock — you now have full access to TinyFish Web Agent for browsing, scraping, and automation. Your immediate focus is activating these credentials in your .env file and running your first TinyFish call.

5. Go to: form.typeform.com/to/wwt0oJ3e
6. Fill in: your name, email, and use case (copy this):

Use Case Text to Submit	Building a Founder Intelligence Briefing Agent using Google ADK + TinyFish Web Agent. The agent uses TinyFish to browse competitor sites, scrape pricing changes, and monitor competitor job boards — returning structured JSON intelligence into a multi-agent daily brief for tech founders. Built in Python using google-adk with AgentTool architecture.

7. Also apply for $2,000 in Google Cloud credits: cloud.google.com/startup (pre-funded category)

3.2 Build Your Demo (3 Days Max)

The accelerator requires a 2-3 minute demo video showing your app in action. Raw and authentic — no slides. Here is exactly what to show:

Timestamp	What to Show	Why It Impresses
0:00–0:30	Open terminal, run `adk web`, open browser — show ADK dev UI loading	Shows technical credibility — you built this from scratch

0:30–1:00	Type 'Give me my morning brief' — show the agent trace firing sub-agents in real time	Multi-agent architecture is visually compelling in ADK's trace panel
1:00–1:40	Show TinyFish actually navigating a competitor site (screen record the browser automation)	TinyFish integration is your killer differentiator — show it browsing
1:40–2:10	Read out the final brief — show structured Revenue + Inbox + Competitor output	The output speaks for itself — founders will immediately want this
2:10–2:30	Say: 'This replaces 2 hours of manual information gathering every morning for tech founders'	Clear problem → solution narrative for investors watching

3.3 Post on X — Required for Application

This is mandatory. Post your demo video on X. Here is the exact post template to use:

Post Template	Built a Founder Intelligence Briefing Agent using @Tiny_Fish + Google ADK. It browses competitor sites in real time, reads my inbox, and delivers a 3-minute morning brief so I don't lose 2 hours of deep work time every day. Full demo: [link to video] #TinyFishAccelerator #BuildInPublic

Tag @Tiny_Fish in the post. This gets you visibility with the TinyFish team before your formal application even arrives.

3.4 Submit Your Formal Application Email

Email: accelerator@tinyfish.ai — Subject: [Application for TinyFish Accelerator]

Email body template:

Email Body	Hi TinyFish team, I'm applying to the accelerator with Founder Intelligence Briefing Agent — an AI chief of staff built on Google ADK + TinyFish Web Agent that gives tech founders a daily intelligence brief in under 3 minutes. TinyFish powers our competitive intelligence module, which browses competitor sites, monitors pricing pages, and tracks hiring signals. My X post with demo: [link]. Happy to jump on a call. — [Your name], [LinkedIn], [X handle]

3.5 What Happens After You Apply

Stage	What Happens	Your Action
Applied	Accepted applicants enter Pre-Flight status immediately	Watch for Discord invite from TinyFish team
Pre-Flight	Get free API keys, private Discord access, weekly mentorship starts	Join Discord — connect with other founders in the cohort
Build Phase (2 weeks)	Partner stack access — ElevenLabs, MongoDB, Composio, AgentOps credits	Add voice delivery (ElevenLabs) and observability (AgentOps) to your agent
Review (2 days)	Final evaluation — Demo Day lineup announced	Polish your demo, prepare your pitch narrative
Demo Day	Pitch live — funding decisions on the spot ($2M seed pool)	Clear ask: here is what we built, here is traction, here is what $X gets us

PHASE 4

After Acceptance — What to Build Next

Once accepted, you get access to the full partner stack. Here is what to add to your agent using accelerator partner tools:

Partner	What to Add to Your Agent	Why
ElevenLabs	Convert the text brief to a 3-minute voice audio clip — delivered as a morning podcast	Founders can listen during their commute — massive UX upgrade
AgentOps	Wrap your agent runs with AgentOps tracing — see every tool call, latency, cost	Shows investors you have observability — this is production-grade thinking
MongoDB	Store daily briefs, enable 'compare to last week' queries, build trend analysis	Persistent memory makes the agent smarter over time
Composio	Add 100+ integrations (HubSpot, Slack, Notion, Linear) in a few lines	Expands your tool surface without writing custom API connectors
Axiom	Log all agent runs, queries, errors — production monitoring from day one	Required for real customers — shows product maturity
Alguna	Add billing and subscription management for your $49/month pricing	Monetize immediately — take payments before Demo Day

BONUS

Choosing Your Product Domain — Honest Advice

Your Situation	You own Unideploy.in. Your product is a Founder Intelligence Briefing Agent. The honest answer: these do not match, and for an accelerator demo that matters.

Why Unideploy.in Does Not Work for This Product

The name 'Unideploy' signals DevOps — CI/CD pipelines, server deployments, infrastructure. When a tech founder or investor sees 'unideploy.in' as the URL for an AI morning brief agent, there is a cognitive mismatch. They expect one thing and get another. For a Demo Day pitch where you have 2 minutes to establish trust, that friction is a real cost.

Recommended Domain Names — Ranked

Domain	Why It Works	Availability Check	Verdict
chiefos.ai	Chief of Staff OS — exactly what the product is. Memorable, category-defining.	Check on Namecheap	Best
founderbrief.ai	Instantly clear — a brief for founders. Zero explanation needed.	Check on Namecheap	Excellent
morningbrief.ai	Describes the core use case (7AM daily brief). Simple and specific.	Check on Namecheap	Very Good
briefai.in	.in keeps India connection, 'briefai' is compact and clear.	Likely available	Good (budget)
agent.unideploy.in	Subdomain on your existing domain — zero cost, usable for MVP only.	You own it already	MVP only

What to Do Right Now

For the accelerator application this week: use agent.unideploy.in as a subdomain. It costs nothing and takes 5 minutes to set up. Point it to a simple landing page on Vercel.

In parallel: go to namecheap.com or godaddy.com and check chiefos.ai or founderbrief.ai today. Good .ai domains get taken fast, especially once you start posting publicly on X with the #TinyFishAccelerator hashtag.

One-Page Landing Page (Required for Accelerator)	You need a URL to put in your X post and application email. Use v0.dev (Vercel's AI UI builder — also a TinyFish accelerator partner) to generate a landing page in 10 minutes. Prompt: 'Build a dark-theme SaaS landing page for an AI agent called [Product Name] that gives tech founders a 3-minute morning intelligence brief. Include headline, 3 feature bullets, and email signup form.'

Minimal Landing Page Setup with Vercel (Free)

```
# 1. Install Vercel CLI
$ npm install -g vercel

# 2. Create a minimal landing page
$ mkdir landing && cd landing
$ touch index.html

# 3. Deploy in one command (no account needed first time)
$ vercel
# → Follow prompts, get a live URL like: https://your-project.vercel.app

# 4. Add custom subdomain (if using agent.unideploy.in)
# In Vercel dashboard: Settings → Domains → Add 'agent.unideploy.in'
# In your domain registrar: add CNAME record pointing to
cname.vercel-dns.com
# Takes ~10 minutes to propagate
```

Quick Reference Checklist

Phase 0 — Done When:

- ✅ Python 3.12 + google-adk installed, venv active
- ✅ .env file with all 4 API keys
- ✅ Project structure created (founder_agent/ folder)

Phase 1 — Done When:

- ✅ revenue_agent.py, inbox_agent.py, competitor_agent.py all created
- ✅ Gmail OAuth done — token.json exists
- ✅ adk web runs — morning brief appears in browser

Phase 2 — Done When:

- ✅ TinyFish competitor browsing working — returns structured JSON
- ✅ Brief delivered via email or cron job

Phase 3 — Done When:

- ✅ TinyFish API credits — CONFIRMED ACTIVE. Key pasted into .env and smoke test passed
- ✅ Google Cloud $2K credits applied for
- ✅ Demo video recorded (2-3 min, raw, shows TinyFish browsing live)
- ✅ X post live with #TinyFishAccelerator #BuildInPublic
- ✅ Email sent to accelerator@tinyfish.ai with X post link

Applications reviewed every Friday.

The next review window is this Friday.

Start Phase 0 today.